Lesson 10 · Testing, TDD & 100% Coverage

xUnit v3, the two test platforms, a 100% coverage gate that fails the build, and the mutation score that tells you whether 100% meant anything

Contents

1 · Why this lesson — you already test; .NET changes the plumbing 2
- 1.1 · The numbers to leave this lesson with 2

2 · The frameworks — and the platform underneath them 2
- 2.1 · Concept map — your test stack in .NET names 3
- 2.2 · Which platform, and what each command then does 4
- 2.3 · Attribute map — one row per thing you write every day 4
- 2.4 · The same parameterised test in JUnit 5 and xUnit v3 4
- 2.5 · The strong suite's project file — VSTest, with the gate imported 5
- 2.6 · The same theory in Visual Basic — and the project file behind it 5

3 · TDD on the no-claim-bonus rule — red, green, refactor 5
- 3.1 · Red — the first test, before the rule exists 6
- 3.2 · What red looks like, and why the exact-type assertion matters 6
- 3.3 · Green — the rule as a switch expression over patterns 7
- 3.4 · The rating sheet the theory asserts — worked by hand first 7
- 3.5 · The rule that refuses — a guard, not a loading 7

4 · Fixtures and isolation — the constructor is your @BeforeEach 8
- 4.1 · The fixture is three readonly fields and a constructor 8
- 4.2 · What the runner does for every single test case 9
- 4.3 · Isolation by framework — the row that breaks inherited suites 9

5 · Assertions and test doubles — one licence decision, two habits 9
- 5.1 · An interaction test — NSubstitute instead of Mockito 10
- 5.2 · Testing a 30-day expiry in C# and in Visual Basic 10
- 5.3 · Assertions and doubles — what to adopt, what to inherit 11

6 · Coverage as a gate — 100% line and branch, failing the build 12
- 6.1 · The gate — one .props file every test project imports 12
- 6.2 · One test deleted, all tests green, build red 13
- 6.3 · What one deleted test costs the gate 13
- 6.4 · Coverage tooling — pick by platform first 13

7 · Beyond coverage — what the mutation score tells you 14
- 7.1 · The 100%-covered test that tests nothing 14
- 7.2 · Same coverage, different suites 15
- 7.3 · Where the mutants went 15
- 7.4 · The life of one mutant 15
- 7.5 · Mutant states, and what the strong suite actually produced 15
- 7.6 · Thresholds that fail the build, next to the coverage ones 16

8 · Integration, API and BDD tests — in memory, with a clock you own 16
- 8.1 · Where an in-memory API test actually runs 17
- 8.2 · POST then GET — the whole pipeline, no port 17
- 8.3 · Replacing one service for one test 18
- 8.4 · Executable specification — Gherkin on the left, its bindings on the right 18
- 8.5 · Which test to write — cost against what it can prove 18

9 · Hands-on — run the suites, then judge them 19
- 9.1 · Commands 19
- 9.2 · What you should see 20
- 9.3 · The 51 test cases by kind 20
- 9.4 · Sample code by role 20
- 9.5 · What carries over, what is new, what to unlearn 21

Page numbers are resolved from the rendered PDF. Code panels are cut from the compiled samples; every score in sections 6 and 7 comes from a real run on the build machine.

1 · Why this lesson — you already test; .NET changes the plumbing

This is the lesson where your existing standards transfer almost intact — and where one of them turns out to be weaker than you thought. You run TDD, you hold 100% unit-test coverage as a delivery standard, you automate Postman collections per environment, you run k6 and you gate merges on SonarQube. None of that has to change. What changes is the plumbing: .NET has *two* test platforms under one framework choice, its coverage gate is written in MSBuild rather than a plugin block, and its most popular assertion library became a paid product for commercial use in 2025.

The standard that needs upgrading is the coverage number itself. This lesson ships two suites against the same library. Both reach **100% line and 100% branch coverage**, so both pass the same gate. One of them asserts real behaviour and kills 43 of 47 injected bugs (91.49%); the other runs every line, swallows exceptions and kills 4 (8.51%). Both numbers are measured, in §7, from a real Stryker.NET run in this repository. After that, "100% covered" stops being the last line of a quality report.

The vocabulary maps cleanly, so most of the work is translation. `@Test` is `[Fact]`, `@ParameterizedTest` is `[Theory]`, Mockito is NSubstitute, MockMvc is `WebApplicationFactory`, JaCoCo is coverlet, PIT is Stryker.NET and Cucumber is Reqnroll. Where a name is the only difference this lesson gives it one row in the map (2.1) and moves on; the pages go to the four places where .NET behaves differently.

You will build 9 projects around the MotorQuote rating rules. One library under test, a strong C# suite, a Visual Basic suite, a deliberately weak suite, two Microsoft.Testing.Platform twins for the mutation runs, Gherkin specifications, a minimal quote API and its in-memory tests. Premiums are illustrative, not a real tariff. lesson 08 (ASP.NET Core Web APIs) owns ASP.NET Core itself; lesson 09 (Data Access with EF Core) owns EF Core; this lesson owns how you test them.

1.1 · The numbers to leave this lesson with

COVERAGE GATE	MUTATION, STRONG SUITE	MUTATION, WEAK SUITE	XUNIT V3	LESSON 10 TESTS
100 / 100	**91.49%**	**8.51%**	**4.0.1**	**51**
line + branch, fails dotnet test · measured	43 of 47 mutants killed · measured	same 100% coverage · measured	12 Sep 2026 · verified	from 25 methods + 2 scenarios · measured

KEY: • Same idea • Renamed • Different • Trap • Verified • Estimate • Measured C# tooling architecture

i Before you start

1. **You need records and interfaces from lesson 03 (Types, OOP & Generics), `async/await` and `TimeProvider` from lesson 05 (Async, Tasks & Concurrency), project files and MSBuild properties from lesson 07 (Solutions, MSBuild & Mono-repos), and the minimal API shape from lesson 08 (ASP.NET Core Web APIs).** TDD, test doubles, the test pyramid and coverage discipline are assumed — this lesson explains their .NET form, never the ideas.
2. **You will build and run 9 projects, all offline.** `L10.Pricing` is the library under test; `L10.Pricing.Tests` is the strong xUnit suite with the coverage gate; `L10.Pricing.VbTests` is the same rules tested from Visual Basic; `L10.Pricing.WeakTests` is the 100%-covered suite that asserts almost nothing; the two `.Mtp` projects are the same test files built as Microsoft.Testing.Platform executables so Stryker.NET can run them; `L10.Pricing.Specs` holds the Gherkin; `L10.QuoteApi` and its tests exercise the API in memory. Nothing opens a port, and nothing needs Docker.
3. **Mutation testing needs one extra tool:** `dotnet tool restore` inside the samples folder installs `dotnet-stryker` 5.0.0 from `dotnet-tools.json`. Without it every other command in 9.1 still works.
4. **Chips:** • Same idea transfers as-is, • Renamed is the same idea under another name, • Different behaves differently, • Trap looks the same and bites. Version, support and licensing facts are • Verified with a link and were checked on 2026-09-21; numbers computed from this repository or captured from a real run are • Measured; judgements are • Estimate. The Java panel in 2.4 is for comparison and is not compiled.

2 · The frameworks — and the platform underneath them

Choosing between xUnit, NUnit and MSTest is the small decision; choosing the test platform under them is the one that changes your build. A test *framework* defines the model you write against. A test *platform* discovers and runs the tests, integrates with the IDE and the CLI, and owns the extension points — and .NET has two of them: the long-lived **VSTest**, and **Microsoft.Testing.Platform** (MTP), which makes a test project a plain executable. Microsoft's own guidance is to pick one per repository and configure everything consistently, because mixing VSTest-based and MTP-based projects in one solution is not a supported scenario (Microsoft.Testing.Platform vs VSTest).

xUnit v3 is where that shift is most visible: a test project is an `Exe`, not a library. v3 test projects are stand-alone executables, so `dotnet run` runs the tests directly (xUnit.net v3 migration). The NuGet package you reference chooses the

platform: `xunit.v3` takes the release's default MTP version, while `xunit.v3.mtp-off` explicitly disables MTP support so the project runs under VSTest — package variants that arrived in v3 build 3.2.0 (xUnit.net v3 on Microsoft Testing Platform). Both suites in this lesson are built twice, once each way, from the same source files. The test projects in the other lessons of this track that have one (02-06, 08, 09, 11 and 12) still reference xUnit v2 (`xunit` 2.9.3): the `[Fact]`, `[Theory]` and `Assert` calls they show read the same in v3, so what changes is the project file - an `Exe` and the `xunit.v3` packages - and lesson 07 publishes the v3 versions for a mono-repo.

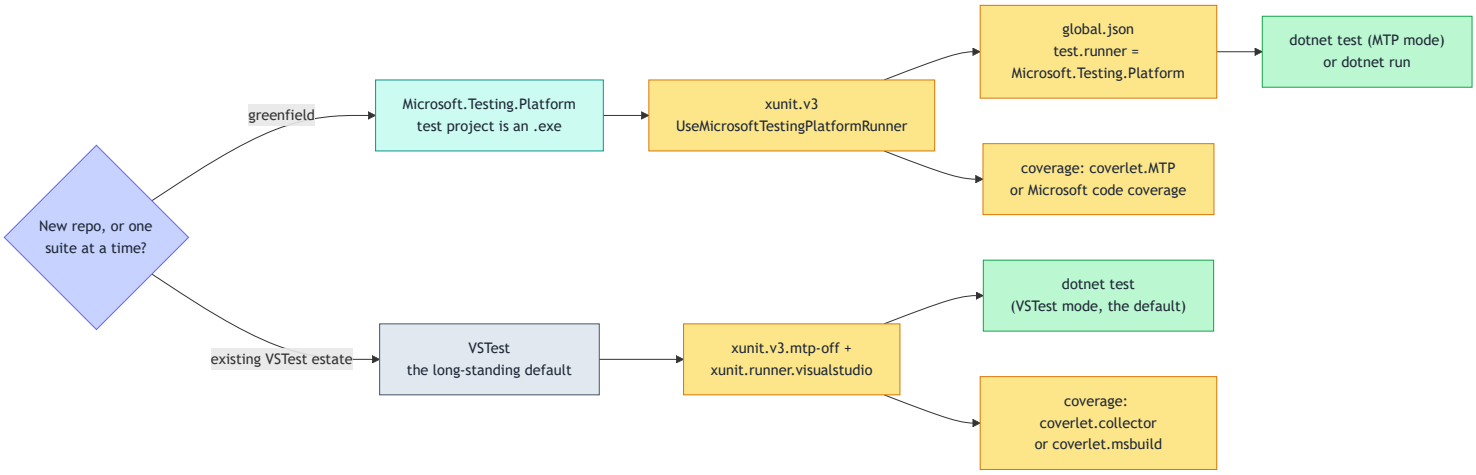
On the .NET 10 SDK, `dotnet test` itself has two modes, and the old bridge between them is being retired. VSTest mode is still the default. MTP mode is new in the .NET 10 SDK and is switched on in `global.json` with `{"test": {"runner": "Microsoft.Testing.Platform"}}`. The legacy path — running an MTP project under VSTest mode via the `TestingPlatformDotnetTestSupport` property — is documented as legacy and will be removed in MTP version 2 on the .NET 10 SDK (dotnet test docs).

All three frameworks are alive and shipping; the differences that matter are behavioural, not syntactic. xUnit v3 4.0.1 (12 Sep 2026), MSTest 4.4.1 (16 Sep 2026) and NUnit 4.6.1 (19 May 2026) are the current releases (xunit.v3 on NuGet · MSTest on NuGet · NUnit on NuGet). Pick xUnit for new work if you have no reason not to — it is the template default and the one the ecosystem samples use. The difference that silently breaks an inherited suite is the test-class lifecycle: xUnit and MSTest build a new instance per test, matching JUnit 5's default, while NUnit shares one instance across a whole fixture (MSTest test lifecycle · NUnit FixtureLifeCycle) — table 4.3 has the rest.

2.1 · Concept map — your test stack in .NET names

You already know	In .NET	Kind	Watch for
JUnit 5 + the Gradle `test` task	xUnit v3 / NUnit / MSTest + `dotnet test`	• Renamed	The platform under the framework is a second, bigger choice (2.2)
`@Test`	`[Fact]`	• Renamed	A fact takes no arguments; anything data-driven is a `[Theory]`
`@ParameterizedTest` + `@MethodSource`	`[Theory]` + `[MemberData]` over `TheoryData<T>`	• Renamed	`TheoryData<T>` is type-checked; a raw `object[]` source is not
`@BeforeEach`	the test class constructor	• Different	xUnit builds a new instance per test case; NUnit shares one (4.3)
Mockito `mock()` / `verify()`	NSubstitute `Substitute.For<T>()` / `Received()`	• Renamed	No `when(...).thenReturn(...)` — you call the member and assign `.Returns()`
A hand-written fake	the same — a real in-memory implementation	• Same idea	`InMemoryQuoteRepository` serves the tests and the API
A `Clock` / `InstantSource` you inject	`TimeProvider` in the BCL	• Renamed	`FakeTimeProvider` is the test double — lesson 05 (Async, Tasks & Concurrency)
MockMvc · `WebTestClient`	`WebApplicationFactory<Program>`	• Same idea	In-process over `TestServer`: no port, no socket (8.1)
Postman collections per environment	`.http` files in the project	• Renamed	Checked in beside the code; secrets come from user-secrets, not the file (8.5)
Cucumber + Gherkin	Reqnroll + the same Gherkin	• Renamed	SpecFlow reached end of life on 31 Dec 2024; Reqnroll is the community fork (8.4)
JaCoCo `jacocoTestCoverageVerification`	coverlet `Threshold` properties	• Renamed	The gate is MSBuild properties in a `.props` file (6.1)
PIT mutation testing	Stryker.NET	• Same idea	Same vocabulary: killed, survived, timeout, mutation score (§7)
Testcontainers for Java	Testcontainers for .NET	• Same idea	Still needs a Docker-compatible runtime, so this lesson does not use it
AssertJ / Hamcrest	built-in `Assert`, AwesomeAssertions or Shouldly	• Trap	Fluent Assertions 8+ needs a paid licence for commercial use (5.3)
`@Disabled("reason")`	`[Fact(Skip = "reason")]`	• Renamed	The reason is the property value, so a skipped test names why
A SonarQube quality gate on coverage	the same, plus a mutation threshold	• Same idea	Coverage is the floor; §7 measures what it misses

2.2 · Which platform, and what each command then does



indigo = the decision · teal = Microsoft.Testing.Platform path · slate = VSTest path · amber = the file or package that selects it · green = what you type

2.3 · Attribute map — one row per thing you write every day

JUnit 5	xUnit v3	NUnit 4	MSTest 4
`@Test`	`[Fact]`	`[Test]`	`[TestMethod]`
`@ParameterizedTest` + `@ValueSource`	`[Theory]` + `[InlineData]`	`[TestCase]`	`[DataRow]`
`@MethodSource`	`[MemberData]` over `TheoryData`	`[TestCaseSource]`	`[DynamicData]`
`@BeforeEach`	the constructor	`[SetUp]`	`[TestInitialize]` or the constructor
`@AfterEach`	`IDisposable.Dispose` / `IAsyncDisposable`	`[TearDown]`	`[TestCleanup]` or `Dispose`
`@BeforeAll`	`IClassFixture` (constructor-injected)	`[OneTimeSetUp]`	`[ClassInitialize]`
`@Disabled`	`[Fact(Skip = "…")]`	`[Ignore]`	`[Ignore]`
`@Tag`	`[Trait("Category", "…")]`	`[Category]`	`[TestCategory]`

2.4 · The same parameterised test in JUnit 5 and xUnit v3

Java — for comparison not compiled — for comparison

```java
// JUnit 5 — the same rule, the same shape
static Stream<Arguments> table() {
    return Stream.of(
        arguments(1, new BigDecimal("0.20")),
        arguments(2, new BigDecimal("0.25")),
        arguments(3, new BigDecimal("0.30")),
        arguments(4, new BigDecimal("0.40")),
        arguments(5, new BigDecimal("0.50")),
        arguments(9, new BigDecimal("0.50")));
}

@ParameterizedTest
@MethodSource("table")
void discountGrowsWithClaimFreeYears(
        int years, BigDecimal expected) {
    var discount = NoClaimBonus.discountFor(years);

    assertThat(discount).isEqualByComparingTo(expected);
}
```

C# samples/L10.Pricing.Tests/NoClaimBonusTests.cs · #region the…

```csharp
public static TheoryData<int, decimal> Table => new()
{
    { 1, 0.20m }, { 2, 0.25m }, { 3, 0.30m },
    { 4, 0.40m }, { 5, 0.50m }, { 9, 0.50m },
};

[Theory]
[MemberData(nameof(Table))]
public void Discount_grows_with_claim_free_years(
    int years, decimal expected)
{
    var discount = NoClaimBonus.DiscountFor(years);

    Assert.Equal(expected, discount);
}
```

> **Same idea, two smaller pieces of ceremony.** A `TheoryData<int, decimal>` is typed, so a wrong column is a compile error rather than a cast failure at run time — the `Stream<Arguments>` on the left is not. The parameter names in the test method must match the data order, not any annotation. And note the money type: `decimal` compares with `==` and with `Assert.Equal`, so there is no `isEqualByComparingTo` to remember — lesson 02 (C# Language Essentials) explains why `decimal` and not `double`.

2.5 · The strong suite's project file — VSTest, with the gate imported

MSBuild / XML samples/L10.Pricing.Tests/L10.Pricing.Tests.csproj

```
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <IsPackable>false</IsPackable>
  </PropertyGroup>

  <Import Project="..\CoverageGate.props" />

  <ItemGroup>
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="18.10.0" />
    <PackageReference Include="xunit.v3.mtp-off" Version="4.0.1" />
    <PackageReference Include="xunit.runner.visualstudio" Version="4.0.0" PrivateAssets="all" />
    <PackageReference Include="NSubstitute" Version="6.2.0" />
    <PackageReference Include="Microsoft.Extensions.TimeProvider.Testing" Version="10.10.0" />
  </ItemGroup>

  <ItemGroup>
    <ProjectReference Include="..\L10.Pricing\L10.Pricing.csproj" />
    <Compile Include="..\Shared\Requests.cs" Link="Shared\Requests.cs" />
    <Using Include="Xunit" />
    <Using Include="L10.Pricing.TestData" />
  </ItemGroup>

</Project>
```

Four lines decide the platform and the tooling. `OutputType Exe` is required by xUnit v3; `xunit.v3.mtp-off` plus `xunit.runner.visualstudio` put the project on VSTest, which is what lets `coverlet.msbuild` in `../CoverageGate.props` measure and gate it (6.1). The `<Compile Include>` line links one shared test-data builder without a project reference, so the builder never counts as production code. The twin project `L10.Pricing.Tests.Mtp` compiles the same `.cs` files with `xunit.v3` and `UseMicrosoftTestingPlatformRunner` instead — and running `dotnet test` on that twin fails immediately with "*Testing with VSTest target is no longer supported by Microsoft.Testing.Platform on .NET 10 SDK and later*", which is exactly the retirement the .NET 10 docs describe. Run it with `dotnet run` (9.1).

2.6 · The same theory in Visual Basic — and the project file behind it

C# samples/L10.Pricing.Tests/NoClaimBonusTests.cs · #region the…

```
public static TheoryData<int, decimal> Table => new()
{
    { 1, 0.20m }, { 2, 0.25m }, { 3, 0.30m },
    { 4, 0.40m }, { 5, 0.50m }, { 9, 0.50m },
};

[Theory]
[MemberData(nameof(Table))]
public void Discount_grows_with_claim_free_years(
    int years, decimal expected)
{
    var discount = NoClaimBonus.DiscountFor(years);

    Assert.Equal(expected, discount);
}
```

Visual Basic samples/L10.Pricing.VbTests/NoClaimBonusVbTests.vb ·…

```
Public Shared ReadOnly Property Table As _
    New TheoryData(Of Integer, Decimal) From {
        {1, 0.2D}, {2, 0.25D}, {3, 0.3D},
        {4, 0.4D}, {5, 0.5D}, {9, 0.5D}
    }

<Theory>
<MemberData(NameOf(Table))>
Public Sub Discount_grows_with_claim_free_years(
        years As Integer, expected As Decimal)
    Dim discount = NoClaimBonus.DiscountFor(years)

    Assert.Equal(expected, discount)
End Sub
```

The VB test is the same xUnit v3 test with different punctuation. `TheoryData(Of Integer, Decimal) From { ... }` is the collection initialiser, `<Theory>` and `<MemberData>` use angle brackets, `Sub` is a `void` method, `Dim` is `var`, and a decimal literal ends in `D` where C# uses `m`. The assertion, the data and the test name are identical, and the VB project runs its 8 cases (this theory's six rows, the throws test and the expiry test in 5.2) under the same `dotnet test`. The one place a VB estate has to get something right is the project file: `L10.Pricing.VbTests.vbproj` sets `OutputType` to `Exe`, which xUnit v3 requires, and references `xunit.v3.mtp-off` plus `xunit.runner.visualstudio` — the same choices as 2.5, with each file saying `Imports Xunit` where the C# project uses a global using. lesson 06 (VB.NET for Legacy Estates) owns VB itself.

3 · TDD on the no-claim-bonus rule — red, green, refactor

The loop is the one you already run; what is worth watching is how much of the design C# hands you for free. The rule: a driver with no claims earns a discount that grows with claim-free years, capped at five years and beyond. Start with the failing test for the zero-years case, write the smallest switch expression that passes, then let the theory table drive the rest of the curve. Illustrative rates, not a real tariff.

Red first, and read the failure message — it is part of the test. The stub throws `NotImplementedException`, so every case fails for the same reason, and the exception-shape test fails with a message that names both types. That message is why `Assert.Throws<T>` matching the exact type is a feature rather than a nuisance: a test that accepts a subclass would have passed against the stub.

Green is a switch expression over patterns, and it is the whole implementation. Relational and constant patterns cover the guard clause, the five steps and the cap in twelve lines. The catch-all arm is what makes every input handled: without it the compiler flags the switch expression as not exhaustive (warning CS8509, an error in these samples because warnings are errors) — lesson 02 (C# Language Essentials) owns pattern matching. The refactor step then has nothing to do, which is the honest outcome for a rule this small; the interesting refactor in this lesson happens in §7, when a mutation run shows that one boundary was never really tested.

Name tests as sentences, not as identifiers. `Negative_years_are_rejected` reads out of a CI log without the reader opening the file. That is the .NET convention you will see in the framework's own repositories, and it is the one the samples in this track follow. Reach for `[Fact(DisplayName = "…")]` only when the sentence will not fit in an identifier.

3.1 · Red — the first test, before the rule exists

C# — samples/L10.Pricing.Tests/NoClaimBonusTests.cs · #region first-test

```
[Fact]
public void No_claim_free_years_means_no_discount()
{
    var discount = NoClaimBonus.DiscountFor(claimFreeYears: 0);

    Assert.Equal(0.00m, discount);
}
```

Three lines and one named argument. `claimFreeYears: 0` is a named argument, not a comment, so the test states which input is zero — the discount, the years and the cap are all numbers and confusing them is the easiest bug in a rating rule. xUnit's `Assert.Equal(0.00m, discount)` compares `decimal` values exactly; nothing here needs a delta or a comparator.

3.2 · What red looks like, and why the exact-type assertion matters

Terminal — the red phase — captured on the build machine

```
# NoClaimBonus.DiscountFor exists but still throws NotImplementedException
dotnet test --filter "FullyQualifiedName~NoClaimBonusTests"

  Failed L10.Pricing.Tests.NoClaimBonusTests.No_claim_free_years_means_no_discount [1 ms]
  Error Message:
   System.NotImplementedException : The method or operation is not implemented.

  Failed L10.Pricing.Tests.NoClaimBonusTests.Negative_years_are_rejected [2 ms]
  Error Message:
   Assert.Throws() Failure: Exception type was not an exact match
Expected: typeof(System.ArgumentOutOfRangeException)
Actual:   typeof(System.NotImplementedException)

Failed!  - Failed:     8, Passed:     0, Skipped:     0, Total:     8, Duration: 43 ms
```

Eight cases fail: one fact, six theory rows and the exception test. The last failure is the one to read — "*Exception type was not an exact match*". `Assert.Throws<ArgumentOutOfRangeException>` rejects the `NotImplementedException` that the stub throws, and it would equally reject a `ArgumentException` base-class throw once the rule is written. If you want the assertion to accept subclasses you have to say so with `Assert.ThrowsAny<T>`. • **Measured** on the build machine from a stub copy of the library.

3.3 · Green — the rule as a switch expression over patterns

C# samples/L10.Pricing/NoClaimBonus.cs · #region ncb-rule

```
public static decimal DiscountFor(int claimFreeYears) =>
    claimFreeYears switch
    {
        < 0 => throw new ArgumentOutOfRangeException(
            nameof(claimFreeYears), "cannot be negative"),
        0 => 0.00m,
        1 => 0.20m,
        2 => 0.25m,
        3 => 0.30m,
        4 => 0.40m,
        _ => 0.50m,
    };
```

The guard clause, the curve and the cap in one expression. `< 0` is a relational pattern and _ is the catch-all that implements "five years and beyond", so the method has no `if` and no fall-through. Every arm is a value, which is why the whole method is an expression-bodied member. Two consequences for testing: the _ arm means no input can reach an unhandled case, and every arm is a branch, so coverlet counts one branch per arm plus the implicit no-match path — 8 for these 7 arms. The theory table in 2.4 walks five of the seven arms; the zero-years fact in 3.1 and the negative-years test in 3.2 cover the other two, and all three are needed to reach 100% branch.

3.4 · The rating sheet the theory asserts — worked by hand first

Scenario	Sum insured · class	Loading	No-claim bonus	Total, THB
Standard — 36, 10 claim-free years	600,000 · Class 1	none	50%	**6,767.96**
Young — 23, one claim, 3 licence years	600,000 · Class 1	+20% age, +10% claim	none	**17,596.71**
Commercial — 45, two claims	400,000 · Class 3	+25% claims, +25% use	none	**2,578.27**
Heavy — 45, 3,500 cc, no claims	400,000 · Class 3	+35% use — over 3,000 cc	50%	**1,160.22**
Boundary — turns 25 on the start date	600,000 · Class 1	none — 25 is not under 25	50%	**6,767.96**
Engine — exactly 3,000 cc, commercial	400,000 · Class 3	+25% use — 3,000 is not over 3,000	50%	**1,074.28**
Track — 23, four claim-free years	550,000 · Class 1	+20% age	40%	**8,933.71**
Declined — 36, three claims	600,000 · Class 1	the tariff refuses, it does not load	—	**no premium**

Every total is net premium + 0.4% stamp duty + 7% VAT on the sum of the two, each rounded to the satang — the canonical tariff the whole track prices against (see the curriculum). The rates are illustrative, not a real tariff; what matters is that every total was worked out from the sheet *before* the code existed. Row 7 is the curriculum's own worked example, so this suite fails the moment this lesson's arithmetic drifts from the other eleven. Rows 5 and 6 are the two boundary cases, and they are why §7 reports every relational mutant dead — they are the rows that catch `< 25` loosened to `<= 25` and `> 3_000` loosened to `>= 3_000`. The last row is not a price at all: three claims in five years is a declined quote, which is 3.5.

3.5 · The rule that refuses — a guard, not a loading

C# samples/L10.Pricing/PremiumCalculator.cs · #region claims-guard

```
public static decimal ClaimsLoading(int claims) =>
    claims switch
    {
        0 => 0.00m,
        1 => 0.10m,
        2 => 0.25m,
        // 3 or more in five years is not a price, it is a refusal
        _ => throw new QuoteDeclinedException(
            "3+ claims in 5 years"),
    };
```

The tariff stops pricing at three claims, so the method has no arm that returns a number for it. The catch-all throws `QuoteDeclinedException`, a two-line type in `Domain.cs` — the smallest thing that carries a reason out of a rating rule, and the form the curriculum's tariff asks for in a lesson that already has an exception vocabulary (lesson 02 (C# Language Essentials) owns exceptions; lesson 09 (Data Access with EF Core) stores the same outcome as `QuoteStatus.Declined`). Two things make this worth its nine lines in a testing lesson: `Three_or_more_claims_is_declined_rather_than_priced` is a `[Theory]` over 3 and 9, so the boundary and the far side are both asserted, and the assertion on `ex.Message` is the one place in this library where a message string is part of the contract — which is why §7 finds its surviving string mutants everywhere except here.

4 · Fixtures and isolation — the constructor is your @BeforeEach

xUnit has no setup attribute because it does not need one: it builds a new instance of the test class for every test case, so the constructor *is* `@BeforeEach` and `readonly` fields *are* your fixture. Teardown is `IDisposable.Dispose` or `IAsyncDisposable.DisposeAsync` on the same class. Nothing can leak from one test into the next through an instance field, which removes a whole class of order-dependent failures before you write any code.

That default is not universal, and the exception is the trap. NUnit creates one instance and shares it with every test in the fixture unless the class is marked `[FixtureLifeCycle(LifeCycle.InstancePerTestCase)]` (NUnit FixtureLifeCycle). A JUnit or xUnit engineer reading an NUnit suite will assume isolation that is not there; a mutable field set in one test is visible in the next, and the suite passes or fails depending on execution order. MSTest, like xUnit and JUnit 5, creates a fresh instance per test (MSTest test lifecycle).

Shared expensive setup is opt-in, and it is injected rather than inherited. `IClassFixture<T>` builds one `T` for the whole test class and hands it to the constructor — that is how 8.2 gets one `WebApplicationFactory<Program>` for three API tests instead of booting the app three times. It is `@BeforeAll` without a static field, so each class gets its own copy even when classes run in parallel (next paragraph). One level up, the assembly-level attribute `[assembly: AssemblyFixture(typeof(T))]` does the same job for a whole assembly — still constructor-injected, but with no interface on the test class (xUnit shared context); NUnit's `[SetUpFixture]` and MSTest's `[AssemblyInitialize]` are the equivalents.

xUnit runs test *classes* in parallel by default, and the constructor does not protect you from that. Tests inside one class run one after another, each on its own instance; two classes run at the same time, because each class is its own test collection by default (xUnit parallelism). Instance fields are safe. A `static` field, a file in the temp folder, an environment variable or a fixed port is shared by every class running at that moment, and JUnit 5 runs sequentially unless you configure it, so a suite that got away with those will flake here. Put the classes that share a resource in one `[Collection]` — they then run one after another — and share the expensive object with an `ICollectionFixture<T>`.

One xUnit v3 habit to adopt on day one: pass the cancellation token. The analyzer xUnit1051 warns whenever you call a method that accepts a `CancellationToken` without giving it one, and the token to give it is `TestContext.Current.CancellationToken` so that a timed-out test cancels its work instead of leaking it (xUnit1051). With warnings-as-errors on, as in every sample here, that warning is a build failure — which is why every asynchronous test in this lesson starts by capturing `ct`.

4.1 · The fixture is three readonly fields and a constructor

C# — samples/L10.Pricing.Tests/QuoteServiceTests.cs · #region fixture

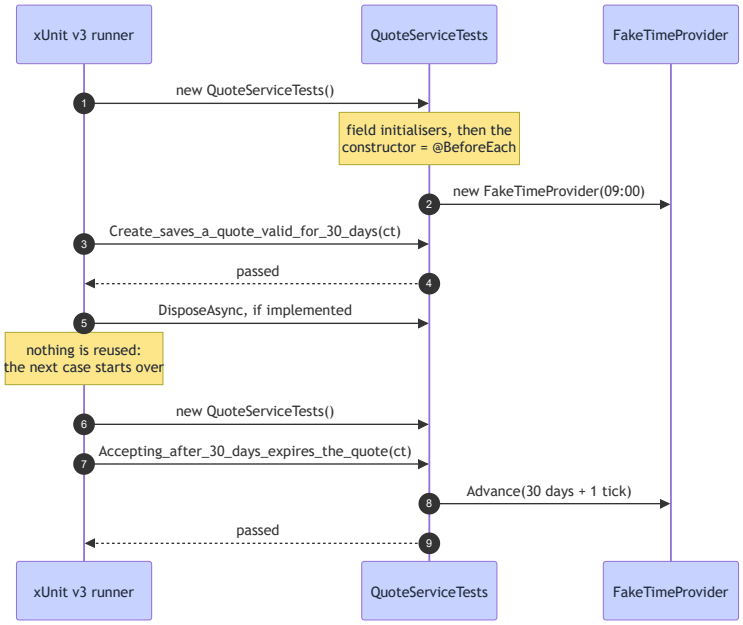
```
// xUnit creates a new instance of this class for every test:
// the constructor is @BeforeEach, and nothing leaks between tests.
private static readonly DateTimeOffset Nine =
    new(2026, 10, 1, 9, 0, 0, TimeSpan.Zero);

private readonly FakeTimeProvider _time = new(Nine);
private readonly IQuoteRepository _repo =
    Substitute.For<IQuoteRepository>();
private readonly QuoteService _service;

public QuoteServiceTests() =>
    _service = new(_repo, new PremiumCalculator(), _time);
```

No attributes, no base class, no field mutation. Each test case gets a fresh `FakeTimeProvider` pinned to 09:00, a fresh NSubstitute double and a fresh `QuoteService`, so a test that advances the clock cannot affect any other. `Nine` is `static readonly` because it is a constant, not state. The comment in the sample says what the shape means; the equivalent NUnit class would need `[SetUp]` and — to get the same isolation — `[FixtureLifeCycle(LifeCycle.InstancePerTestCase)]`.

4.2 · What the runner does for every single test case



one construction, one test, one optional disposal — then the whole cycle repeats for the next case, with a brand-new instance

4.3 · Isolation by framework — the row that breaks inherited suites

Framework	Instance per test case?	Per-test setup	Watch for
xUnit v3	● Yes — always	the constructor	There is no setup attribute to look for; read the constructor
MSTest 4	● Yes	`[TestInitialize]` or the constructor	Constructor first, then `[TestInitialize]` (MSTest test lifecycle)
NUnit 4	● No — one shared instance	`[SetUp]`	Add `[FixtureLifeCycle(LifeCycle.InstancePerTestCase)]` or keep every field immutable (NUnit FixtureLifeCycle)
JUnit 5	● Yes — PER_METHOD	`@BeforeEach`	The habit you already have; xUnit and MSTest match it, NUnit does not

5 · Assertions and test doubles — one licence decision, two habits

Start with the assertions the framework ships, because the fluent library you would reach for by reflex is now a procurement question. Fluent Assertions — the AssertJ of .NET — moved to the Xceed Community License at version 8: commercial users must buy a per-seat annual subscription, while version 7 stays Apache-2.0 and keeps receiving fixes (Fluent Assertions FAQ). Pinning v7, adopting the community fork AwesomeAssertions or writing built-in `Assert` calls are all defensible; adding a paid dependency to every test file by accident is not. Put it in the licence scan your pipelines already run.

For test doubles, .NET's default answer is NSubstitute, and the reason is partly historical. Moq is older and more widespread, but version 4.20.0 shipped SponsorLink, which scanned local git data, and it was removed the next day in 4.20.2 (Moq changelog). The episode moved a lot of teams to NSubstitute, whose API is also closer to what you want: you call the member and assign a return value, instead of wrapping it in a `when(...)`.

Prefer a fake for state and a substitute for interaction. When a test asks "was it saved?", a substitute with `Received(1)` answers exactly that. When a test asks "what happens after it is saved?", a real in-memory implementation is simpler, faster to read and impossible to over-specify — which is why `L10.QuoteApi` keeps one instead of a database: the subject here is the test, and lesson 09 (Data Access with EF Core) already showed the persistent version of the same quote store, a `MotorQuoteDbContext` with a `DbSet<Quote>`. Over-mocked tests fail on a rename; state-based ones fail on a behaviour change.

Never read the clock in production code: take a `TimeProvider`. It is in the base class library, so it needs no abstraction of your own, and `FakeTimeProvider` from `Microsoft.Extensions.TimeProvider.Testing` lets a test move time by an exact `TimeSpan` (FakeTimeProvider). That is how 5.2 tests a 30-day expiry in a millisecond, in both languages — lesson 05 (Async, Tasks & Concurrency) owns `TimeProvider` itself.

5.1 · An interaction test — NSubstitute instead of Mockito

C# samples/L10.Pricing.Tests/QuoteServiceTests.cs · #region nsubstitute

```
[Fact]
public async Task Create_saves_a_quote_valid_for_30_days()
{
    var ct = TestContext.Current.CancellationToken;
    var request = Requests.Standard();

    var quote = await _service.CreateAsync(request, ct);

    Assert.Equal(Nine.AddDays(30), quote.ValidUntil);
    await _repo.Received(1).SaveAsync(
        Arg.Is<Quote>(q => q.Status == QuoteStatus.Quoted),
        ct);
}
```

Two assertions of two different kinds. `Assert.Equal` checks the value the service returned; `Received(1)` checks that it saved exactly once, with a `Quoted` quote and the same cancellation token. `Arg.Is<Quote>(q => ...)` is Mockito's `argThat`. Note the `await` before `_repo.Received(1)`: verifying an async member is itself awaited, and forgetting it is the commonest NSubstitute mistake.

5.2 · Testing a 30-day expiry in C# and in Visual Basic

C# samples/L10.Pricing.Tests/QuoteServiceTests.cs · #region fak…

```
[Fact]
public async Task Accepting_after_30_days_expires_the_quote()
{
    var ct = TestContext.Current.CancellationToken;
    var quote = await _service.CreateAsync(
        Requests.Standard(), ct);
    _repo.FindAsync(quote.Id, ct).Returns(quote);

    _time.Advance(QuoteService.Validity); // still valid
    _time.Advance(TimeSpan.FromTicks(1)); // one tick late
    var result = await _service.AcceptAsync(quote.Id, ct);

    Assert.Equal(QuoteStatus.Expired, result.Status);
}
```

Visual Basic samples/L10.Pricing.VbTests/QuoteServiceVbTests.vb ·…

```
Private Shared ReadOnly Nine As New DateTimeOffset(
    2026, 10, 1, 9, 0, 0, TimeSpan.Zero)

<Fact>
Public Async Function Expired_after_30_days() As Task
    Dim ct = TestContext.Current.CancellationToken
    Dim time As New FakeTimeProvider(Nine)
    Dim repo = Substitute.For(Of IQuoteRepository)()
    Dim service As New QuoteService(
        repo, New PremiumCalculator(), time)

    Dim quote = Await service.CreateAsync(Request(), ct)
    repo.FindAsync(quote.Id, ct).Returns(quote)
    time.Advance(QuoteService.Validity) ' still valid
    time.Advance(TimeSpan.FromTicks(1)) ' one tick late
    Dim result = Await service.AcceptAsync(quote.Id, ct)

    Assert.Equal(QuoteStatus.Expired, result.Status)
End Function
```

The same test, the same libraries, one runtime. Both advance the fake clock by exactly the validity window and then by a single tick, so the assertion proves the boundary is exclusive rather than approximately right — a real clock could never test this. The VB side shows what a VB test project costs you: `Async Function ... As Task` instead of `async Task` and `Dim ... As New` instead of target-typed `new`, but the same `<Fact>` attribute, the same `Substitute.For(Of T)()` and the same `Assert.Equal`. A legacy VB estate can be brought under test without a rewrite first — lesson 06 (VB.NET for Legacy Estates).

5.3 · Assertions and doubles — what to adopt, what to inherit

library view

1 Built-in Assert — start here

C# · Same idea

xUnit's `Assert` class: equality, collections, exceptions, async.

YOU KNOW	JUnit's `Assertions` before AssertJ arrived
STRENGTH	`Assert.Equal` on a `record` compares by value, so one call checks a whole object
THROWS	`Assert.Throws<T>` is exact-type by default — a subclass fails the assertion (3.2 shows the message)
LICENCE	Apache-2.0 with the framework, nothing to decide

2 Fluent Assertions 8+ — a licence decision

tooling · Trap

The AssertJ of .NET, and since version 8 a paid product for commercial use.

CHANGED	v8 moved to the Xceed Community License; commercial users must buy a per-seat annual subscription (Fluent Assertions FAQ)
V7	Stays Apache-2.0 and gets fixes — pinning it is a supported answer
DO NOW	Treat it like any other licence finding in the pipeline scan you already run
WATCH	It reads like an assertion style; it is a procurement item

3 AwesomeAssertions · Shouldly

tooling · Renamed

The community answers to that change, both free for commercial use.

FORK	AwesomeAssertions is the community fork of Fluent Assertions 7, same API (AwesomeAssertions)
OTHER	Shouldly is an older, smaller alternative with a different style
COST	Swapping assertion library touches every test file — decide once, per repo
SAMPLE	This lesson uses built-in `Assert` only, so nothing to swap

5.3 · Assertions and doubles (continued)

library view

4 NSubstitute — the default mock

C# · Renamed

`Substitute.For<T>()`, then call the member to arrange or assert.

MOCKITO	`when(x.y()).thenReturn(z)` becomes `x.Y().Returns(z)`
VERIFY	`await repo.Received(1).SaveAsync(...)` replaces `verify(repo, times(1))`
MATCHERS	`Arg.Any<T>()` and `Arg.Is<T>(p => ...)`
LICENCE	BSD-3-Clause, 6.2.0 · Aug 2026 (NSubstitute on NuGet)

5 Moq — read it, inherit it

tooling · Different

The oldest and most common mocking library; you will meet it in existing code.

SHAPE	`new Mock<T>()`, `.Setup(...)`, `.Object`, `.Verify(...)`
HISTORY	4.20.0 (8 Aug 2023) shipped SponsorLink, which scanned local git data; 4.20.2 removed it the next day (Moq changelog)
TODAY	Fine technically; many teams moved anyway and never moved back
ADVICE	Do not mix two mocking libraries in one repo

6 Fakes and FakeTimeProvider

architecture · Same idea

A real implementation with a cheap backing store, and a clock you control.

FAKE	`InMemoryQuoteRepository` is used by the tests *and* by the API
CLOCK	`FakeTimeProvider` ships in `Microsoft.Extensions.TimeProvider.Testing` (FakeTimeProvider)
MOVES	`Advance(TimeSpan)`, `SetUtcNow(...)`, `AutoAdvanceAmount`
PREFER	A fake for state, a substitute for interaction — 5.1 substitutes the repository, and the API tests in §8 run against the real in-memory one

6 · Coverage as a gate — 100% line and branch, failing the build

In .NET the coverage gate is MSBuild properties, not a plugin block, and that is good news: it lives in one `.props` file that every test project imports. coverlet is the de-facto tool. `Threshold`, `ThresholdType` and `ThresholdStat` make `dotnet test` itself fail below the bar, so the gate runs on a developer's machine and in CI with no extra step and nothing to configure twice (coverlet).

Gate both line and branch, because line coverage alone is satisfied by walking through a condition without ever taking the other side of it. 6.2 deletes exactly one test — the one that passes an undefined enum value — and coverage falls to 98.86% line and 97.14% branch. Every remaining test passes, and the command still fails. That is the behaviour you want: a suite cannot quietly shrink.

Excluding code from the measurement is a discipline, not a convenience. `[ExcludeFromCodeCoverage]` applies to an assembly, class, struct, constructor, method, property or event, and it carries a `Justification` property — use it, because an unexplained exclusion is how a 100% gate becomes decoration (ExcludeFromCodeCoverageAttribute). The honest targets are generated code and thin composition roots; a hand-written branch you cannot reach is a design problem, not a coverage problem.

One migration trap deserves its own sentence: your coverage driver depends on the platform you chose in §2. coverlet's own documentation states that `coverlet.collector` and `coverlet.msbuild` cannot be used with Microsoft.Testing.Platform, because both rely on VSTest infrastructure; the MTP driver is `coverlet.MTP` (coverlet.MTP), and Microsoft ships its own code-coverage extension for MTP as well (Microsoft.Testing.Platform code coverage). Move a suite to MTP without changing the driver and the gate does not fail — it simply stops existing.

6.1 · The gate — one .props file every test project imports

MSBuild / XML — `samples/CoverageGate.props`

```
<Project>
  <!-- The 100% gate. Imported by every test project that must hold L10.Pricing at
       100% line AND branch coverage. coverlet.msbuild instruments the assembly when
       `dotnet test` runs (VSTest mode) and fails the command below the threshold. -->
  <PropertyGroup>
    <CollectCoverage>true</CollectCoverage>
    <Include>[L10.Pricing]*</Include>
    <Threshold>100</Threshold>
    <ThresholdType>line,branch</ThresholdType>
    <ThresholdStat>total</ThresholdStat>
    <CoverletOutputFormat>cobertura</CoverletOutputFormat>
    <CoverletOutput>$(ArtifactsPath)/coverage/$(MSBuildProjectName)/</CoverletOutput>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="coverlet.msbuild" Version="10.0.1" PrivateAssets="all" />
  </ItemGroup>
</Project>
```

Seven properties are the whole policy. `Include=[L10.Pricing]*` measures the library under test and nothing else, so test-support code cannot inflate the number; `Threshold 100` with `ThresholdType line,branch` fails the command below either bar; `ThresholdStat total` applies it to the whole module rather than per-file. Cobertura is the output format because every report tool reads it — ReportGenerator turns it into browsable HTML and a badge (ReportGenerator). Importing this file rather than repeating the properties is the lesson 07 (Solutions, MSBuild & Mono-repos) pattern: one place to raise the bar for every suite.

6.2 · One test deleted, all tests green, build red

Terminal — the gate doing its job captured on the build machine

```
# one test deleted: the (CoverageClass)99 arm of BaseRate is no longer reached
dotnet test L10.Pricing.Tests

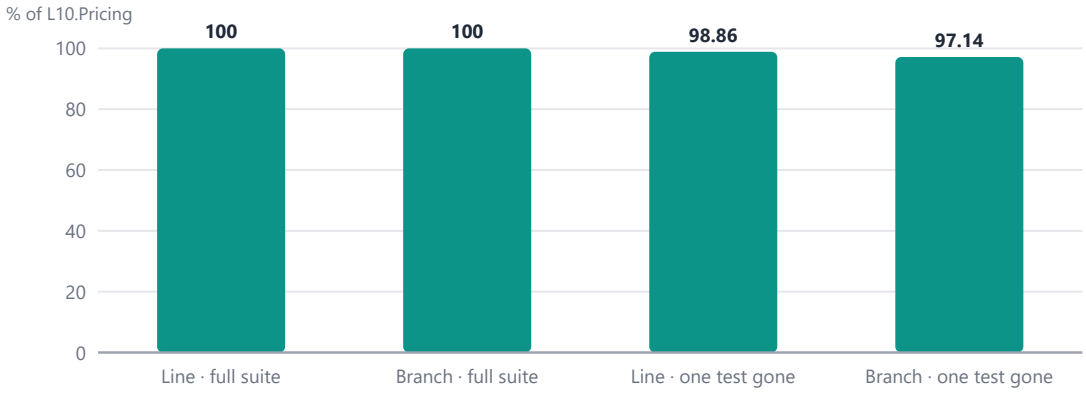
Passed!  - Failed: 0, Passed: 32, Skipped: 0, Total: 32, Duration: 92 ms
  [coverlet]
  Calculating coverage result...
   Generating report '.build/artifacts/coverage/L10.Pricing.Tests/coverage.cobertura.xml'

+-------------+--------+--------+--------+
| Module      | Line   | Branch | Method |
+-------------+--------+--------+--------+
| L10.Pricing | 98.86% | 97.14% | 100%   |
+-------------+--------+--------+--------+

coverlet.msbuild.targets(73,5): error : The total line coverage is below the specified 100
coverlet.msbuild.targets(73,5): error : The total branch coverage is below the specified 100
# every test passed and dotnet test still failed — that is the gate
```

98.86% line and 97.14% branch — and `dotnet test` fails. The deleted test was the one that passes `(CoverageClass)99`, so the `_ => throw` arm of `BaseRate` is never reached: one line and one branch. Branch coverage falls further than line coverage because there are far fewer branch points than lines — 35 against 88 in that run — so one uncovered branch costs 2.86 points and one uncovered line 1.14, which is exactly why gating both is worth the extra property. • Measured on the build machine from a scratch copy of the samples with that one test removed; the committed suite holds 100 / 100 / 100.

6.3 · What one deleted test costs the gate



coverlet totals for the L10.Pricing module · the configured threshold is 100 for both line and branch · measured from two real runs on the build machine

The gap looks tiny and that is the point: a threshold of 100 turns a 1.14-point drop into a build failure. A gate at 95% would have accepted 98.86% line and 97.14% branch without a word. Read the two right-hand bars as the cost of one deleted test in a library of 120 lines — then read §7, which shows that the bars being back at 100 still does not mean the code is tested. • Measured

6.4 · Coverage tooling — pick by platform first

Tool	Platform	Output	Use it for
`coverlet.msbuild`	• VSTest only	cobertura, opencover, json, lcov	**The gate in this lesson** — thresholds that fail `dotnet test`
`coverlet.collector`	• VSTest only	same formats	The template default; collects coverage but is driven by run settings
`coverlet.MTP`	• MTP	json + cobertura by default	The MTP driver — what you move to when you leave VSTest (coverlet.MTP)
`coverlet.console`	• Either	same formats	A global tool for one-off runs and non-`dotnet test` harnesses
Microsoft code coverage extension	• MTP	coverage (default), xml, cobertura	Microsoft's own MTP coverage extension; pass `--coverage-output-format cobertura` if a report tool has to read it (Microsoft.Testing.Platform code coverage)
ReportGenerator	• Either	HTML, badges, summaries	Turning a cobertura file into something a human or a PR comment can read (ReportGenerator)

7 · Beyond coverage — what the mutation score tells you

100% coverage says every line ran. It does not say that anything was checked — and this lesson ships the proof as a compiling project. `L10.Pricing.WeakTests` has two test methods. It loops over every input, swallows the exceptions with a `try/catch`, calls `Assert.NotNull` twice and asserts nothing else. It reaches **100% line and 100% branch coverage** and it passes the same gate as the strong suite's 33 cases.

Mutation testing is the measurement that separates them. Stryker.NET rewrites your production code one small change at a time — a comparison flipped, an operator swapped, a statement removed, a string emptied — and runs the tests against each mutant. A mutant that makes a test fail is *killed*; one that leaves every test passing *survived*, and it is a bug your suite would not catch. The score is `detected / valid × 100` (mutant states and metrics · Stryker.NET). This is PIT, with the same vocabulary.

The measured result: 91.49% against 8.51%, from identical coverage. Of 47 tested mutants the strong suite kills 43 and the weak suite kills 4. The weak suite's survivors are not exotic: `< 25` becomes `> 25`, `loading += 0.20m` becomes `-=`, `ClaimsLast5Years == 0` becomes `!= 0` so every careful driver loses the no-claim bonus, `net * 0.004m` becomes a division, a whole statement disappears. Every one of those would ship a wrong premium, and every one is invisible to a coverage report.

Read your survivors before you chase the number. All 4 survivors in the strong suite are string mutations inside exception messages — the text of `"cannot be negative"`, of `"unknown class"` in `BaseRate` and of two interpolated `$"quote {id} ..."` messages. Nothing asserts on that text, and for internal exceptions nothing should — while the one message this library *does* assert, the decline reason in 3.5, is killed. That is the whole rule in miniature: a string is worth mutating only where somebody reads it. The useful response is to assert the message where a caller depends on it, exclude message mutations otherwise, and set the threshold below 100 on purpose — the configuration in 7.6 breaks the build under 80%, which the weak suite promptly does.

7.1 · The 100%-covered test that tests nothing

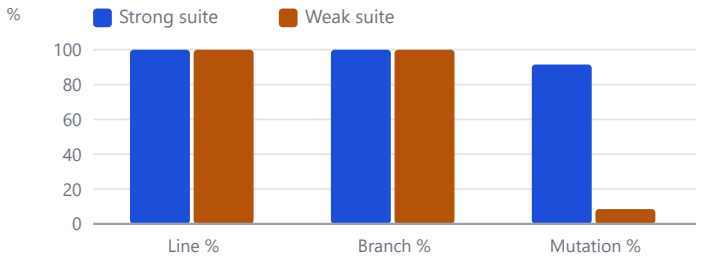
C# — samples/L10.Pricing.WeakTests/WeakSuite.cs · #region weak

```
[Fact]
public void Every_rating_rule_runs()
{
    for (var years = -1; years <= 5; years++)
        Swallow(() => NoClaimBonus.DiscountFor(years));
    for (var claims = 0; claims <= 3; claims++)
        Swallow(() => PremiumCalculator.ClaimsLoading(claims));
    foreach (var coverage in Enum.GetValues<CoverageClass>())
        PremiumCalculator.BaseRate(coverage);
    Swallow(() => PremiumCalculator.BaseRate((CoverageClass)99));
    PremiumCalculator.AgeAt(new DateOnly(2001, 10, 2), Requests.Start);

    var calculator = new PremiumCalculator();
    Swallow(() => calculator.Calculate(null!));
    Assert.NotNull(calculator.Calculate(Requests.Standard()));
    Assert.NotNull(calculator.Calculate(Requests.Standard(
        age: 23, claims: 1, use: VehicleUse.Commercial)));
    Assert.NotNull(calculator.Calculate(Requests.Standard(
        claims: 2, use: VehicleUse.Commercial, engineCc: 3_500)));
}
```

Every line of the rating rules runs, and only two assertions ever fire. `Swallow` wraps each throwing call in a `try/catch`, so the guard clauses are *covered* without anyone checking which exception came out; the loop from `-1` to `5` walks every arm of the no-claim-bonus switch without checking a single discount. Do not copy this pattern — it exists so that 7.2 can put a number on it. If a suite you inherit is full of `Assert.NotNull` and empty catch blocks, you are looking at this.

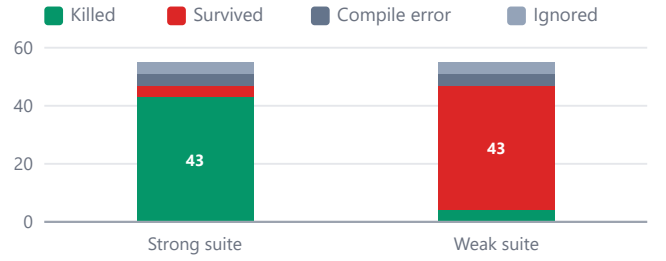
7.2 · Same coverage, different suites



coverlet totals and Stryker.NET scores for the same library · measured from real runs on the build machine

The first two pairs are identical; the third is a 82.98-point gap. A gate that reads only the first two bars calls both suites finished. • Measured

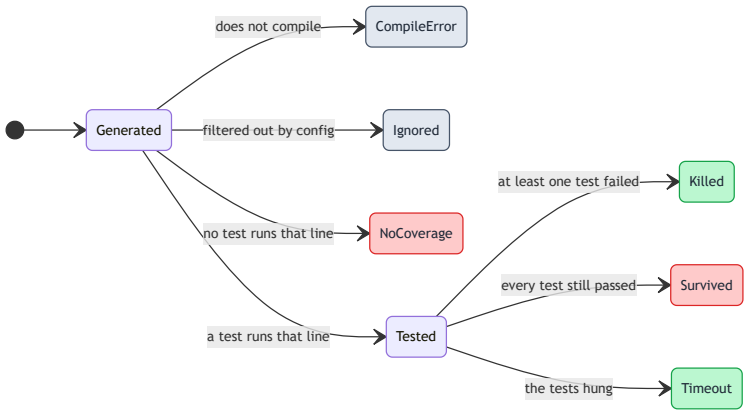
7.3 · Where the mutants went



one Stryker.NET run per suite over the same generated mutants · green = detected, red = counted against you, slate = excluded from the score, as in 7.4

47 mutants are scored; 8 are not. A mutant that does not compile and one filtered out as already-covered are both excluded from `detected / valid`, which is why the denominator is 47 and not 55 (mutant states and metrics). • Measured

7.4 · The life of one mutant



green = counted as detected · red = counted against you · slate = excluded from the score entirely · lavender = still in flight · the score is detected / valid

7.5 · Mutant states, and what the strong suite actually produced

State	What it means	In the score?	Strong suite
Killed	At least one test failed while the mutant was active	• detected	**43**
Survived	Every test passed — a bug your suite cannot see	• undetected	**4** — all of them exception-message strings
No coverage	No test reaches that line at all	• undetected	**0** — the 100% gate guarantees this
Timeout	The run hung, for example an inverted loop condition	• detected	**0**
Compile error	The mutated code does not build	• excluded	**4**
Ignored	Filtered out — here, mutants inside a block already covered	• excluded	**4**

Only the first four states are scored. Definitions from mutant states and metrics; the counts are • Measured from the Stryker.NET JSON report of the run in 9.2.

7.6 · Thresholds that fail the build, next to the coverage ones

JSON — samples/L10.Pricing.Tests.Mtp/stryker-config.json

```
{
  "stryker-config": {
    "test-runner": "mtp",
    "concurrency": 1,
    "reporters": ["progress", "cleartext", "html", "json"],
    "thresholds": { "high": 90, "low": 80, "break": 80 }
  }
}
```

`break: 80` is the line that makes this a gate rather than a report. Below it Stryker exits non-zero — the weak suite's 8.51% ends with "*Final mutation score is below threshold break. Crashing...*". `high` and `low` only colour the HTML report. `test-runner: mtp` is why the `.Mtp` twin projects exist: Stryker drives the tests through Microsoft.Testing.Platform. `concurrency: 1` keeps the run deterministic on a laptop; raise it in CI. Stryker.NET's default coverage analysis runs only the tests that cover each mutant (Stryker.NET configuration), so a run is far cheaper than one suite execution per mutant — 14 seconds for the strong run and 12 for the weak one (• **Measured** , one run each, concurrency 1, projects already built). The cost still grows with the library and the suite, so run it nightly or per pull request on changed projects rather than on every commit (• **Estimate** for that cadence).

8 · Integration, API and BDD tests — in memory, with a clock you own

`WebApplicationFactory<Program>` is MockMvc with the real pipeline: your whole application, hosted in the test process, reached through an `HttpClient` that never opens a socket. The `Microsoft.AspNetCore.Mvc.Testing` package copies the app's dependency file, sets the content root to the application project and boots it on `TestServer`; `CreateClient()` hands you a client that follows redirects and keeps cookies (integration tests in ASP.NET Core). Middleware, model binding, routing, filters and JSON options all run — which is the point, because those are the layers a unit test cannot reach.

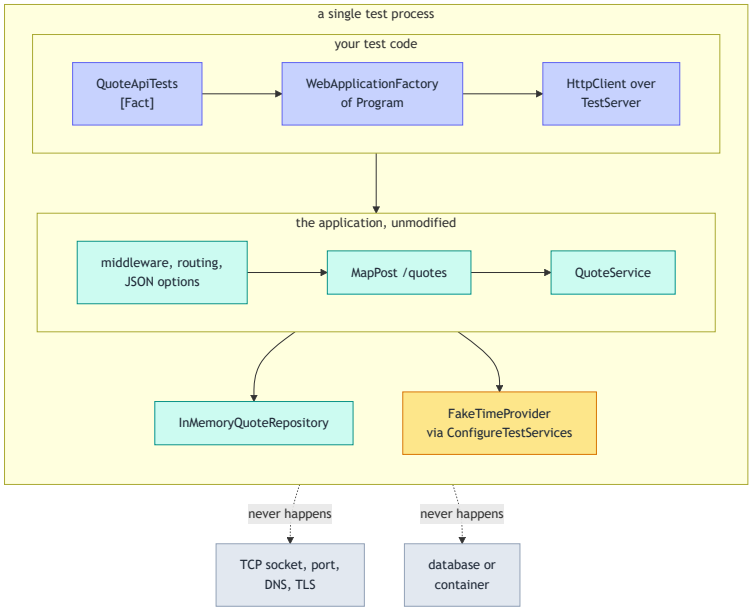
On .NET 10 you no longer have to make `Program` public yourself. Top-level statements compile to an `internal Program` class, which used to force either a hand-written `public partial class Program { }` or an `InternalsVisibleTo` attribute. ASP.NET Core now ships a source generator that emits that declaration for apps using top-level statements; it landed for the 10.0 release (public Program source generator). `L10.QuoteApi/Program.cs` contains no such class, and `WebApplicationFactory<Program>` in the test project compiles — • **Measured** by the fact that the project builds with warnings as errors.

The technique that makes in-memory API tests worth writing is replacing one service. `WithWebHostBuilder` plus `ConfigureTestServices` re-registers a single dependency for one test — here a `FakeTimeProvider` in place of `TimeProvider.System` — so the test can create a quote, move the world 31 days forward and assert a 409. No database, no container, no sleep. Replace a partner client the same way to test a timeout path.

Out-of-process tools stay in the toolbox, and BDD earns its keep only at the boundary. `.http` files are checked-in request collections with variables and response chaining — the sample's third request reads the quote id out of the first response, which is what a Postman collection does minus the export step (.http files). Testcontainers gives you a real database in a throwaway container when an in-memory double would lie, at the cost of a Docker runtime (Testcontainers for .NET), and Playwright for .NET drives real browsers (Playwright .NET). Gherkin earns its ceremony when a non-engineer reads or writes the examples, and is pure overhead when the only readers are its authors.

Across an estate of services the pyramid keeps its shape and gains one boundary: the partner. Each service keeps many fast unit tests and a handful of in-process API tests like the ones above, and cannot prove a partner it does not own from inside its own process. So test each outbound client against a stub of the boundary — a hand-written `HttpMessageHandler` that returns a canned body, a 503 or a delay (lesson 08 (ASP.NET Core Web APIs) owns `HttpClientFactory` and its resilience policies), or WireMock.Net when you want a real stub HTTP server inside the test (WireMock.Net) — and the retry, timeout and failure paths are proved without calling anyone. Where you can get the other side to verify it, a consumer-driven contract test such as Pact.Net pins the shape both ways (Pact.Net); for a third-party partner you cannot ask, the stubbed boundary is the test you have. Browser and container layers stay few. This is a design judgement (• **Estimate**), not a measurement.

8.1 · Where an in-memory API test actually runs



one process, one boundary crossed · indigo = test code · teal = the application under test, unmodified · amber = the one service the test replaces · slate = what is absent

8.2 · POST then GET — the whole pipeline, no port

C# `samples/L10.QuoteApi.Tests/QuoteApiTests.cs · #region round-trip`

```
[Fact]
public async Task Post_then_get_returns_the_same_quote()
{
    var ct = TestContext.Current.CancellationToken;
    using var client = factory.CreateClient();

    var created = await client.PostAsJsonAsync("/quotes", Requests.Standard(), Json, ct);
    var quote = await created.Content.ReadFromJsonAsync<Quote>(Json, ct);
    var fetched = await client.GetFromJsonAsync<Quote>(created.Headers.Location, Json, ct);

    Assert.Equal(HttpStatusCode.Created, created.StatusCode);
    Assert.Equal(6_767.96m, quote!.Premium.Total.Amount);
    Assert.Equal(quote.Id, fetched!.Id);
}
```

One client, three HTTP calls, three assertions. `PostAsJsonAsync` and `GetFromJsonAsync` serialise with a client-side copy of the options the API is configured with — the `HttpClient` never sees `ConfigureHttpJsonOptions`, so keeping the two in step is your job, and a drift in naming policy or enum handling fails here rather than in production. The `GET` reuses `created.Headers.Location` instead of rebuilding the URL, which makes the test also an assertion about the `Created` response. The factory itself arrives through `IClassFixture` (§4), so all three tests in the class share one booted application.

8.3 · Replacing one service for one test

C# samples/L10.QuoteApi.Tests/QuoteApiTests.cs · #region swap-clock

```
[Fact]
public async Task Accepting_an_expired_quote_returns_409()
{
    var ct = TestContext.Current.CancellationToken;
    var clock = new FakeTimeProvider(new DateTimeOffset(2026, 10, 1, 9, 0, 0, TimeSpan.Zero));
    await using var app = factory.WithWebHostBuilder(host =>
        host.ConfigureTestServices(services => services.AddSingleton<TimeProvider>(clock)));
    using var client = app.CreateClient();

    var created = await client.PostAsJsonAsync("/quotes", Requests.Standard(), Json, ct);
    var quote = await created.Content.ReadFromJsonAsync<Quote>(Json, ct);
    clock.Advance(TimeSpan.FromDays(31));
    var accepted = await client.PostAsync($"/quotes/{quote!.Id}/accept", null, ct);

    Assert.Equal(HttpStatusCode.Conflict, accepted.StatusCode);
}
```

`ConfigureTestServices` runs after the application's own registrations, so the last `AddSingleton<TimeProvider>` wins. The test then owns time: create a quote at 09:00, advance 31 days, and the accept call must answer **409 Conflict** because the 30-day validity has passed. Note `await using` on the derived factory — it owns a second host that must be disposed. This is the hook to reach for whenever a test needs a different partner client, a stub authority or a deterministic random source; lesson 11 (Security & Identity with Entra ID) uses it to sign test tokens.

8.4 · Executable specification — Gherkin on the left, its bindings on the right

Gherkin — NoClaimBonus.feature samples/L10.Pricing.Specs/Featur…

```
Feature: No-claim bonus
  Careful drivers pay less. The track's canonical tariff,
  illustrative rates, not a real tariff.

  Background:
    Given a private car insured for 600000 THB, Class 1

  Scenario Outline: Claim-free years earn a discount
    Given a driver aged 36 with <years> claim-free years
    When the premium is calculated
    Then the net premium is <net> THB

    Examples:
      | years | net      |
      | 0     | 12600.00 |
      | 1     | 10080.00 |
      | 3     | 8820.00  |
      | 5     | 6300.00  |

  Scenario: A claim cancels the bonus and adds a loading
    Given a driver aged 36 with 10 claim-free years
    And the driver made 1 claim in the last 5 years
    When the premium is calculated
    Then the net premium is 13860.00 THB
```

C# samples/L10.Pricing.Specs/Steps/PremiumSteps.cs · #region st…

```
[Given("a private car insured for {decimal} THB, Class 1")]
public void GivenACar(decimal sumInsured) =>
    _sumInsured = new Money(sumInsured, "THB");

[Given("a driver aged {int} with {int} claim-free years")]
public void GivenADriver(int age, int years) =>
    (_dateOfBirth, _claimFreeYears) =
        (Start.AddYears(-age), years);

[Given("the driver made {int} claim(s) in the last 5 years")]
public void GivenClaims(int claims) => _claims = claims;

[When("the premium is calculated")]
public void WhenCalculated() =>
    _premium = new PremiumCalculator().Calculate(Request());

[Then("the net premium is {decimal} THB")]
public void ThenNet(decimal net) =>
    Assert.Equal(net, _premium!.Net.Amount);
```

Reqnroll generates one xUnit test per scenario and per example row, so this feature file is five test cases. `{int}` and `{decimal}` are Cucumber expressions, not regular expressions — the parameters arrive typed, which is why `ThenNet` takes a `decimal` and compares money exactly. `Background` runs before every scenario, as in Cucumber. SpecFlow, the library this API comes from, reached end of life on 31 December 2024 after Tricentis discontinued it; Reqnroll is the community fork that carried the project forward, and it is what new .NET work uses (SpecFlow EOL).

8.5 · Which test to write — cost against what it can prove

Layer	In this lesson	Costs	Reach for it when
Unit	`L10.Pricing.Tests` · `L10.Pricing.VbTests`	milliseconds, no I/O	A rule, a boundary or an error path — the bulk of the suite
Executable spec	`L10.Pricing.Specs` (Reqnroll)	a feature file plus bindings to maintain	A non-engineer reads or writes the examples — rating tables, eligibility rules
In-process API	`L10.QuoteApi.Tests` (`WebApplicationFactory`)	a host boot per class, still no port	Serialisation, routing, status codes, middleware, DI wiring

Layer	In this lesson	Costs	Reach for it when
Container-backed	not used here — no Docker on the build machine	a container per fixture, and a Docker runtime	Provider-specific SQL or a broker where an in-memory double would lie (Testcontainers for .NET)
Partner boundary	described only — a stub handler is a few lines, WireMock.Net a package	no network, one handler or stub to write	Proving retry, timeout and failure paths against ~30 partners without calling them (WireMock.Net)
Out-of-process HTTP	`L10.QuoteApi.http`	needs the app running; not part of `dotnet test`	Exploring, demonstrating, or replacing a Postman collection (.http files)
Browser / E2E	mentioned only — no lesson in this track builds one	slowest and most brittle	A user journey no lower layer can prove (Playwright .NET)
Mutation	`*.Mtp` + Stryker.NET	only the covering tests run per mutant — 12–14 seconds here	Judging the suite itself, nightly or per pull request

9 · Hands-on — run the suites, then judge them

Four commands, in this order: prove the 9 projects build and pass, watch the coverage gate, run the same tests on the other platform, then score both suites. The first command is the one to keep in muscle memory — `verify_samples.py` builds every project, runs every test project and runs every console app to completion.

The third command is the platform lesson from §2 in one line. `L10.Pricing.Tests.Mtp` compiles the same test files as a Microsoft.Testing.Platform executable, so you run it with `dotnet run` and get a different output shape — a `Test run summary` block instead of VSTest's `Passed!` line. Running `dotnet test` against it fails by design on the .NET 10 SDK, because this repository does not opt its `global.json` into MTP mode.

The fourth command is the one that changes how you report quality. Two Stryker runs, same library, same 100% coverage: 91.49% and 8.51%. The second run exits non-zero and says so. Put that number beside the coverage number in whatever dashboard your team already reads.

9.1 · Commands

Terminal

```
# 1 · build, test and run all nine projects
python 0-script/verify_samples.py --only 10

# 2 · the strong suite with its 100% line + branch gate (VSTest mode)
dotnet test lesson-10-testing-tdd-coverage/samples/L10.Pricing.Tests

# 3 · the same tests as a Microsoft.Testing.Platform executable
dotnet run --project lesson-10-testing-tdd-coverage/samples/L10.Pricing.Tests.Mtp

# 4 · mutation-test the strong suite, then the deliberately weak one
cd lesson-10-testing-tdd-coverage/samples
dotnet tool restore
cd L10.Pricing.Tests.Mtp     && dotnet stryker
cd ../L10.Pricing.WeakTests.Mtp && dotnet stryker
```

Nothing here needs a network, a database or Docker. `dotnet test` restores and builds first. `dotnet tool restore` reads `samples/dotnet-tools.json` and installs `dotnet-stryker` 5.0.0 locally; Stryker must be started from inside the test project's folder, which is why step 4 changes directory. Expect each mutation run to take twelve to fifteen seconds on a warm build at `concurrency: 1` (• **Measured** once on the build machine); a cold restore and build take longer.

9.2 · What you should see

Output — the four commands captured on the build machine · sample paths shortened

```
# 1 · verify_samples.py --only 10   (paths shortened, five PASS lines left out)
dotnet SDK 10.0.401  (D:\_env_storeage\dotnet\dotnet.exe)
PASS  library     6.9s  L10.Pricing/L10.Pricing.csproj
PASS  test        6.0s  L10.Pricing.Tests/L10.Pricing.Tests.csproj
PASS  console     5.3s  L10.Pricing.Tests.Mtp/L10.Pricing.Tests.Mtp.csproj
PASS  web         5.0s  L10.QuoteApi/L10.QuoteApi.csproj

9/9 sample projects passed

# 2 · dotnet test L10.Pricing.Tests
Passed!  - Failed: 0, Passed: 33, Skipped: 0, Total: 33, Duration: 91 ms
| Module      | Line | Branch | Method |
| L10.Pricing | 100% | 100%   | 100%   |

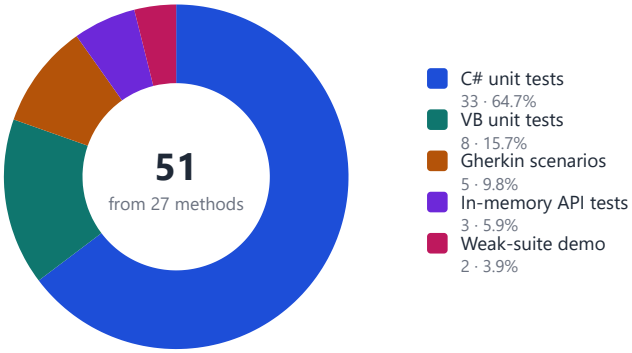
# 3 · dotnet run --project L10.Pricing.Tests.Mtp
Test run summary: Passed! - L10.Pricing.Tests.Mtp.dll (net10.0|x64)
  total: 33   failed: 0   succeeded: 33   skipped: 0   duration: 306ms

# 4 · dotnet stryker, in each .Mtp project
[INF] 47 total mutants will be tested
Killed:   43      Survived:  4      Timeout:   0
[INF] The final mutation score is 91.49 %
Killed:    4      Survived: 43      Timeout:   0
[INF] The final mutation score is 8.51 %
[WRN] Final mutation score is below threshold break. Crashing...
```

Your timings will differ; the counts and the two scores should not. Read the four blocks as one argument: everything passes, the strong suite holds 100 / 100 / 100, the same 33 tests also pass on the other platform with no source change, and then the mutation scores split the two suites apart. The last line is Stryker failing the weak suite against its own `break: 80` threshold — a non-zero exit you can put in a pipeline.

• Measured

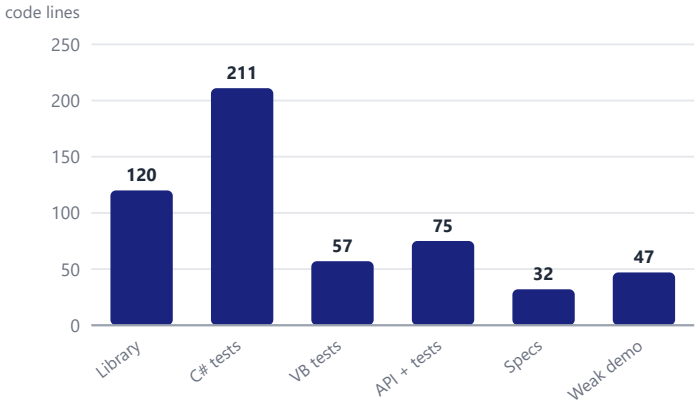
9.3 · The 51 test cases by kind



test cases reported by the runs in 9.2 · one Gherkin example row and one [Theory] row each count as a case

25 hand-written test methods and 2 Gherkin scenarios (27 test methods once Reqnroll has generated its two) produce 51 cases, and most of the multiplication is table data. Counting cases rather than methods is what makes a `TheoryData` table worth writing: one method, six rows, six independent failures with their own names in the log — and the same holds for the four example rows of the one Scenario Outline in 8.4. • Measured

9.4 · Sample code by role



non-blank, non-comment lines · measured from the sample files when this PDF was built

396 lines of test code to 120 lines of library — about 3.3:1. That ratio is normal for a rules engine held at 100% branch coverage, and it is the number to quote when someone asks what a coverage standard costs. Line counts are a size signal, not a quality score.

• Measured

9.5 · What carries over, what is new, what to unlearn

Carries over unchanged

- Red / green / refactor, and naming tests as sentences
- The test pyramid and where to spend the budget
- Gating merges on coverage, and reporting it
- Fakes over mocks for state, doubles for interaction
- Gherkin — the same syntax, a different runner

Learn fresh

- Two test platforms, and which one your tooling supports
- The coverage gate as MSBuild properties in a shared `.props`
- `TimeProvider` / `FakeTimeProvider` instead of your own clock
- `TestContext.Current.CancellationToken` in every async test
- Mutation score as a second, harder gate

Unlearn

- "100% coverage means the code is tested" — §7 measures the gap
- "Setup goes in an annotated method" — in xUnit it is the constructor
- "The test class instance is reused" — true in NUnit, false in xUnit and MSTest
- "A fluent assertion library is a style choice" — since v8 it is a licence
- "Integration tests need a server and a port" — they need neither

⚠ Traps

1. **Isolation is not what JUnit 5 gave you, in two ways.** NUnit shares one fixture instance across its tests unless the class carries `[FixtureLifeCycle(LifeCycle.InstancePerTestCase)]`, so a mutable field becomes cross-test state and the result depends on execution order (NUnit FixtureLifeCycle · 4.3). And xUnit runs test classes in parallel by default: a static, a temp file or a shared port that JUnit 5's sequential default let you get away with will flake — put the classes that share a resource in one `[Collection]` (xUnit parallelism · §4).
2. **Your test platform decides which tools still work, and the two platforms must not be mixed.** `coverlet.collector` and `coverlet.msbuild` cannot run under Microsoft.Testing.Platform: move a suite from VSTest to MTP without switching to `coverlet.MTP` or Microsoft's coverage extension and the threshold does not fail — it stops being evaluated — so check that your gate still *fails* after a migration. Microsoft documents mixing VSTest and MTP projects in one solution as unsupported, and on the .NET 10 SDK `dotnet test` in VSTest mode refuses an MTP project outright. Pick one platform per repository (coverlet.MTP · Microsoft.Testing.Platform vs VSTest · dotnet test docs · 6.4).
3. **Fluent Assertions 8 and later is not free for commercial use.** It is the Xceed Community License with a per-seat annual subscription for business use; version 7 remains Apache-2.0. A routine dependency bump can therefore create a licence obligation across every test project (Fluent Assertions FAQ · 5.3).
4. **A passing suite at 100% coverage can still be worthless.** The measured weak suite in this lesson holds 100% line and branch coverage and kills 8.51% of mutants. Coverage answers "did it run"; only an assertion — or a mutation score — answers "was it checked" (§7).
5. **Forgetting the cancellation token is a build error here, not a warning.** xUnit1051 fires on any call that accepts a `CancellationToken` without one, and every sample in this repository treats warnings as errors. Pass `TestContext.Current.CancellationToken` (xUnit1051).

✓ Checkpoint — you are ready for lesson 11 when

1. `python 0-script/verify_samples.py --only 10` reports 9/9 passed on your machine, and `dotnet test` on `L10.Pricing.Tests` prints 100% line and 100% branch.
2. You can say which package reference and which project property put a test project on VSTest rather than on Microsoft.Testing.Platform, and what `dotnet test` does in each mode on the .NET 10 SDK.
3. You can delete one test from the strong suite, predict which line and which branch stop being covered, and explain why the command fails even though every remaining test passes.
4. You can run both Stryker configurations, read a survived mutant in the HTML report and decide whether it deserves a new test or an exclusion.
5. You can write an in-memory API test that replaces one registered service, and say why `WebApplicationFactory<Program>` compiles without a `public partial class Program` on .NET 10.

🔍 Check yourself

1. A test project references `xunit.v3.mtp-off` and `xunit.runner.visualstudio`. Which platform is it on, which coverlet package can measure it, and what has to change to move it to the other platform?
2. Why does `dotnet test` fail on `L10.Pricing.Tests.Mtp` on the .NET 10 SDK, and what are your two ways to run those tests?
3. xUnit has no `[SetUp]` attribute. Where does per-test setup live, what disposes it, and which of the four frameworks in 4.3 would leak state between tests?
4. Two suites both report 100% line and 100% branch coverage of the same library. What single measurement separates them, what is its formula, and what did it measure here?
5. All 4 survivors in the strong suite are string mutations in exception messages. Name one case where that is worth a new test and one where it is not.
6. Which two lines of `CoverageGate.props` would you change to gate at 95% line coverage only, and what would the run in 6.2 then report?
7. You need a test that proves a quote expires after 30 days. Why is `FakeTimeProvider` the answer rather than a `Thread.Sleep`, a shortened validity constant, or a mocked `DateTime.UtcNow`?

Lesson 10 in one line: the framework choice is small (xUnit v3, NUnit, MSTest all work) and the **platform** choice is not — VSTest or Microsoft.Testing.Platform decides what `dotnet test` does and which coverage driver works. Write the gate as MSBuild properties so `dotnet test` fails below 100% line *and* branch, then measure what the gate cannot see: two suites here share 100% coverage and score **91.49%** and **8.51%** on mutation. Fake the clock, fake the repository, host the API in the test process, and keep Gherkin for the examples a non-engineer reads.
Next: lesson 11 (Security & Identity with Entra ID) — locking the same API down with JWT bearer tokens, Entra ID and policy-based authorization, and testing the 401 / 403 / 200 matrix.